



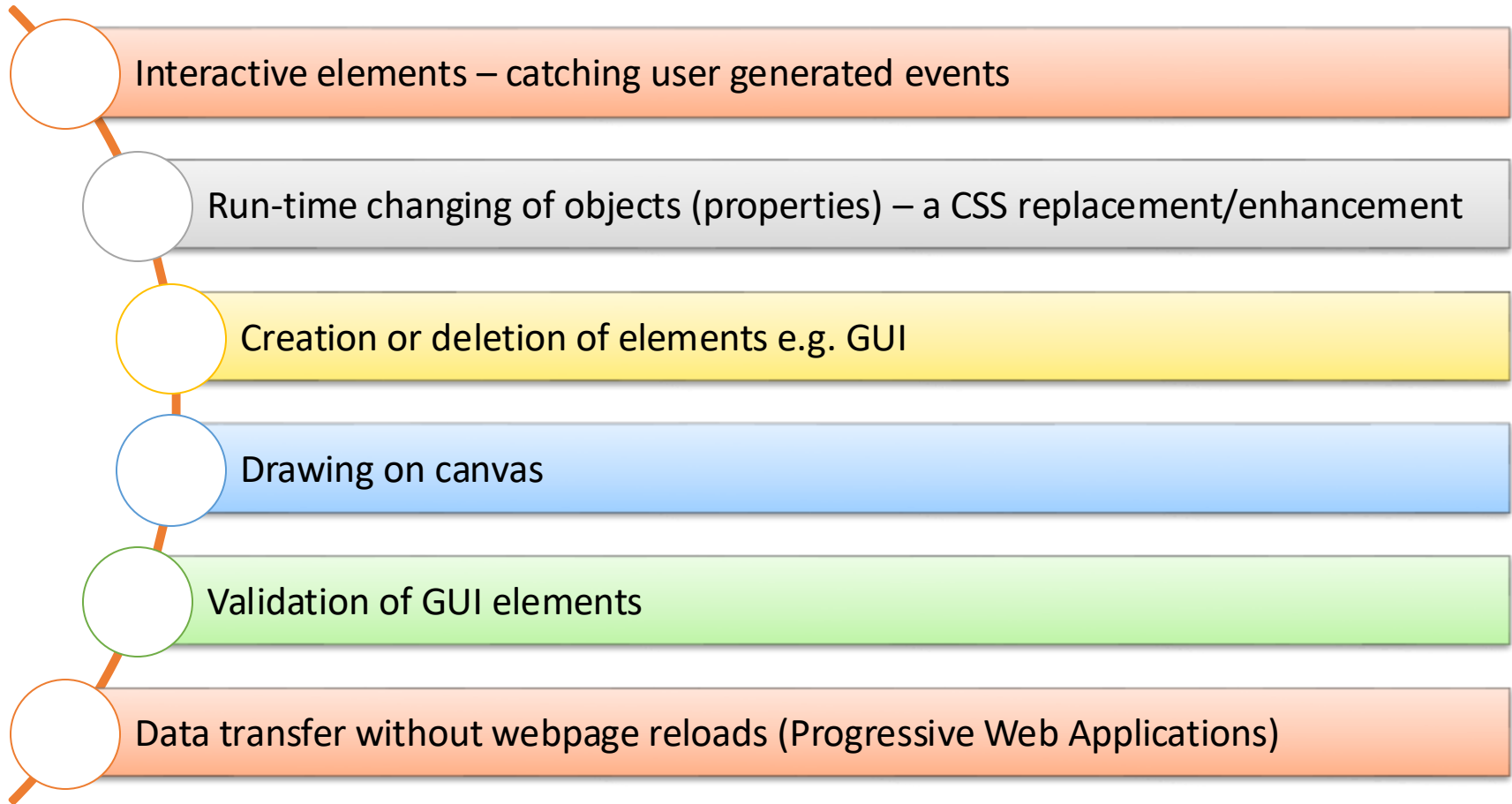
WIESŁAW PIETRUSZKIEWICZ

JAVASCRIPT

JS History

- JavaScript was created by Netscape
- Standardised later ECMA to ECMAScript
- Originally limited to web browsers
- Currently used to deliver various apps and on server side
- A flexible & dynamic language – pros & cons at the same time
- The essential part of modern web browser

JavaScript Common Web Usage



JS – Grammar and types (MDN)

- JavaScript is case-sensitive and uses the Unicode character set.
- `const Früh = "foobar";`
- The syntax of comments is the same as in C++ and in many other languages:
 - `//` a one line comment
 - `/*` this is a longer,
 - `*` multi-line comment
 - `*/`

JS – Declarations (MDN)

- JavaScript has three kinds of variable declarations:
 - `var` - Declares a variable, optionally initializing it to a value.
 - `let` - Declares a block-scoped, local variable, optionally initializing it to a value.
 - `const` - Declares a block-scoped, read-only named constant.

JS -

- The latest ECMAScript standard defines eight data types:
 - Seven data types that are primitives:
 - `Boolean`. `true` and `false`.
 - `null`. A special keyword denoting a null value. (Because JavaScript is case-sensitive, `null` is not the same as `Null`, `NULL`, or any other variant.)
 - `undefined`. A top-level property whose value is not defined.
 - `Number`. An integer or floating point number. For example: 42 or 3.14159.
 - `BigInt`. An integer with arbitrary precision. For example: 9007199254740992n.
 - `String`. A sequence of characters that represent a text value. For example: "Howdy".
 - `Symbol`. A data type whose instances are unique and immutable.
 - and `Object`

JS – Control flow (MDN)

- The most basic statement is a block statement, which is used to group statements. The block is delimited by a pair of curly brackets:

```
{  
    statement1;  
    statement2;  
    // ...  
    statementN;  
}  
  
while (x < 10)  
{  
    x++;  
}
```

JS – Conditional statements (MDN)

- A conditional statement is a set of commands that executes if a specified condition is true. JavaScript supports two conditional statements:

```
if (condition1) {  
    statement1;  
} else if (condition2) {  
    statement2;  
} else if (conditionN) {  
    statementN;  
} else {  
    statementLast;  
}  
  
switch (expression) {  
    case label1:  
        statements1;  
        break;  
    case label2:  
        statements2;  
        break;  
    // ...  
    default:  
        statementsDefault;  
}
```

JS – Loops and iteration

- The statements for loops provided in JavaScript are:
 - for statement
 - do...while statement
 - while statement
 - labeled statement
 - break statement
 - continue statement
 - for...in statement
 - for...of statement

JS – Loops and iteration (MDN)

```
for ([initialExpression]; [conditionExpression]; [incrementExpression])  
    statement
```

```
do  
    statement  
while (condition);
```

```
while (condition)  
    statement
```

```
label:  
    statement
```

```
break;  
break label;
```

```
continue;  
continue label;
```

JS – Loops and iteration (MDN)

```
for (variable in enumerable_object)  
    statement
```

```
for (variable of iterable_object)  
    statement
```

JS – Functions (MDN)

- A function definition (also called a function declaration, or function statement) consists of the function keyword, followed by:
 - The name of the `function`.
 - A list of parameters to the function, enclosed in parentheses and separated by commas.
 - The JavaScript statements that define the function, enclosed in curly brackets,
`{ /* ... */ }`.
- While the function declaration above is syntactically a statement, functions can also be created by a function expression.
- Such a function can be anonymous; it does not have to have a name.

JS – Function example (MDN)

```
function myFunc(theObject) {  
    theObject.make = 'Toyota';  
}  
  
const mycar = {  
    make: 'Honda',  
    model: 'Accord',  
    year: 1998,  
};  
  
// x gets the value "Honda"  
const x = mycar.make;  
  
// the make property is changed by the function  
myFunc(mycar);  
// y gets the value "Toyota"  
const y = mycar.make;
```

JS – Function example (MDN)

```
const square = function (number) {  
  return number * number;  
}
```

```
const x = square(4); // x gets the value 16
```

- Closures allows for the nesting of functions and grants the inner function full access to all the variables and functions defined inside the outer function

```
const pet = function (name) { // Defines a variable called "name"  
  const getName = function () {  
    // The inner function has access to the "name" variable of the outer function  
    return name;  
  }  
  return getName; // Return the inner function, thereby exposing it to outer scopes  
}
```

```
const myPet = pet('Vivie');
```

```
myPet(); // Returns "Vivie"
```

JS – Operators (MDN)

Name	Shorthand operator	Meaning
Assignment	<code>x = f()</code>	<code>x = f()</code>
Addition assignment	<code>x += f()</code>	<code>x = x + f()</code>
Subtraction assignment	<code>x -= f()</code>	<code>x = x - f()</code>
Multiplication assignment	<code>x *= f()</code>	<code>x = x * f()</code>
Division assignment	<code>x /= f()</code>	<code>x = x / f()</code>
Remainder assignment	<code>x %= f()</code>	<code>x = x % f()</code>
Exponentiation assignment	<code>x **= f()</code>	<code>x = x ** f()</code>
Left shift assignment	<code>x <<= f()</code>	<code>x = x << f()</code>
Right shift assignment	<code>x >>= f()</code>	<code>x = x >> f()</code>
Unsigned right shift assignment	<code>x >>>= f()</code>	<code>x = x >>= f()</code>
Bitwise AND assignment	<code>x &= f()</code>	<code>x = x & f()</code>
Bitwise XOR assignment	<code>x ^= f()</code>	<code>x = x ^ f()</code>
Bitwise OR assignment	<code>x = f()</code>	<code>x = x f()</code>
Logical AND assignment	<code>x &&= f()</code>	<code>x && (x = f())</code>
Logical OR assignment	<code>x = f()</code>	<code>x (x = f())</code>
Nullish coalescing assignment	<code>x ??= f()</code>	<code>x ?? (x = f())</code>

JavaScript Example (MDN)

```
/* The first example */
const myHeading = document.querySelector('h1');
myHeading.textContent = 'Hello world!';

/* The second example */
let myImage = document.querySelector('img');

myImage.onclick = function() {
    let mySrc = myImage.getAttribute('src');
    if(mySrc === 'images/firefox-icon.png') {
        myImage.setAttribute('src', 'images/firefox2.png');
    } else {
        myImage.setAttribute('src', 'images/firefox-icon.png');
    }
}
```

JavaScript Objects – MDN Web Docs

- Prototype programming
- A JS traditional approach to design
- Fits better to a dynamic language – you can (re)define objects and create new ones

```
var myCar = new Object();  
myCar.make = 'Ford';  
myCar.model = 'Mustang';  
myCar.year = 1969;  
/* or */  
var myCar = {  
    make: 'Ford',  
    model: 'Mustang',  
    year: 1969  
};
```

JavaScript Classes – MDN Web Docs

```
class Rectangle {  
  constructor(height, width) {  
    this.height = height;  
    this.width = width;  
  }  
  // Getter  
  get area() {  
    return this.calcArea();  
  }  
  // Method  
  calcArea() {  
    return this.height * this.width;  
  }  
}  
  
const square = new Rectangle(10, 10);  
  
console.log(square.area); // 100
```

- Constructor pattern
- Introduced with ECMAScript 6
- Brings JS closer to other “classic” OO-languages (Java, C++)

Embedding JavaScript

Internal JavaScript

- Used for smaller scripts
- JS code is mixed with HTML – difficult maintenance
- Simpler structure

```
<script>  
// JavaScript goes here  
</script>
```

External JavaScript

- Typically used for the majority webpages
- JS code is separated from HTML structure – easier maintenance
- Does not add weight to page

- Note: defer is optional

```
<script src="script.js" defer>  
A message to display if script  
cannot be loaded  
</script>
```

Browser JS Engine

V8

- Used by Google Chrome & Chromium-based browsers (e.g. Microsoft Edge, Opera)

SpiderMonkey

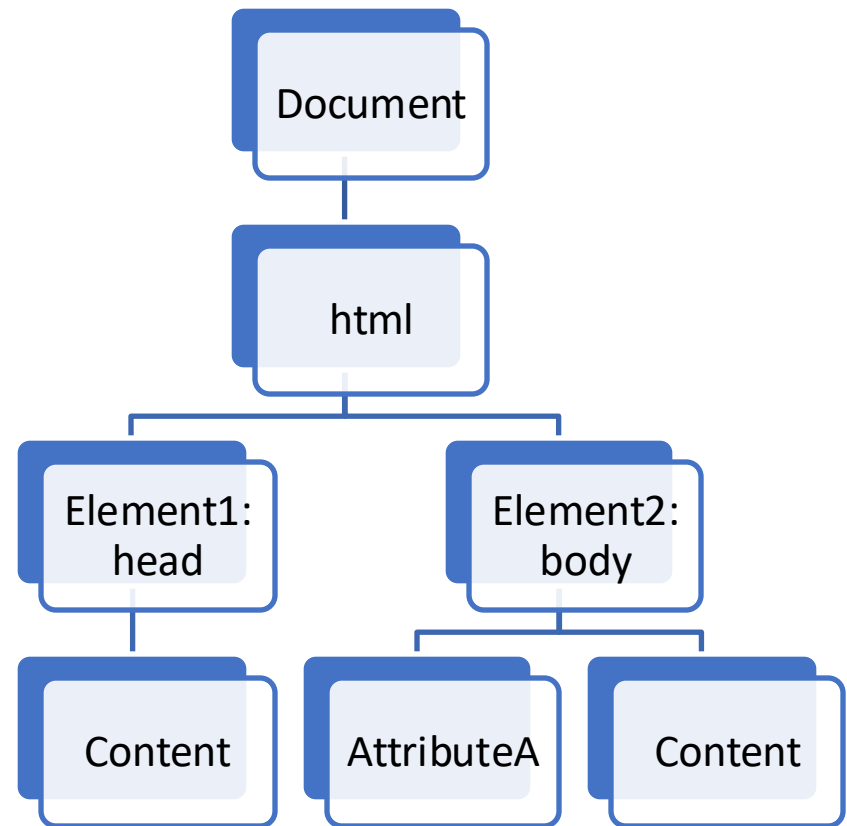
- Used by Mozilla Firefox & other its forks

JavaScriptCore

- Used by Apple Safari (derived from KDE's KJS) & other WebKit browsers (PlayStation, Tizen, Kindle)

DOM

- Document Object Model is a standard structure representing elements constituting a webpage
- A common way used to access & modify elements and their properties



JSON – TBC

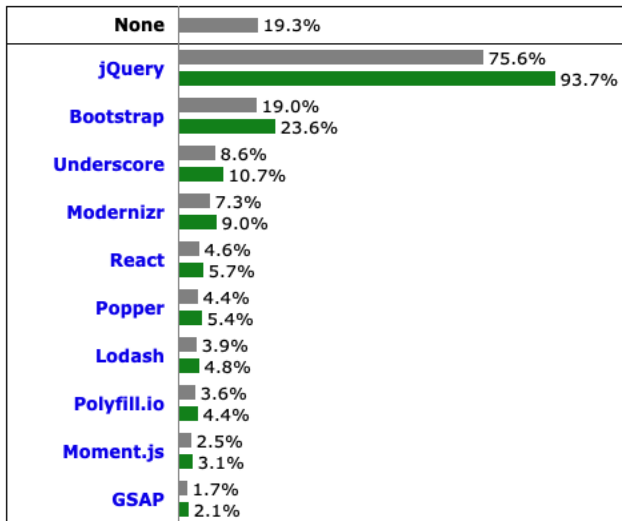
```
{
  "glossary": {
    "title": "example glossary",
    "GlossDiv": {
      "title": "S",
      "GlossList": {
        "GlossEntry": {
          "ID": "SGML",
          "SortAs": "SGML",
          "GlossTerm": "Standard Generalized Markup Language",
          "Acronym": "SGML",
          "Abbrev": "ISO 8879:1986",
          "GlossDef": {
            "para": "A meta-markup language, used to create markup languages such as DocBook.",
            "GlossSeeAlso": ["GML", "XML"]
          },
          "GlossSee": "markup"
        }
      }
    }
  }
}
```

COMMON JAVASCRIPT LIBRARIES

Dynamic Documents and Front-end Web Development

Popularity of JS libraries

https://w3techs.com/technologies/overview/javascript_library



Top 10

jQuery	•A fast, small, and feature-rich JavaScript library that simplifies HTML document traversal, event handling, and animation.
Bootstrap	•A front-end framework for developing responsive and mobile-first websites, providing pre-designed components and a grid system.
Underscore	•A utility library that provides functional programming helpers for working with arrays, objects, and functions in JavaScript.
Modernizr	•A library that detects HTML5 and CSS3 features in the user's browser, enabling developers to create responsive and adaptive designs.
React	•A library for building user interfaces, particularly single-page applications, using a component-based architecture.
Popper	•A library for managing poppers (tooltips, dropdowns, etc.) in web applications, providing positioning and visibility management.
Lodash	•A utility library that provides helpful functions for manipulating arrays, objects, and strings, enhancing JavaScript's capabilities.
Polyfill.io	•A service that detects browser features and serves polyfills to ensure compatibility with older browsers for modern JavaScript features.
Moment.js	•A library for parsing, validating, manipulating, and formatting dates and times in JavaScript, simplifying date handling.
GSAP (GreenSock Animation Platform)	•A high-performance JavaScript library for creating complex animations and transitions in web applications.

JavaScript and Other JS-oriented Technologies

jQuery

- *“jQuery is a fast, small, and feature-rich JavaScript library. It makes things like HTML document traversal and manipulation, event handling, animation, and Ajax much simpler with an easy-to-use API that works across a multitude of browsers.”*
- A Swiss army knife of JavaScript – *de facto* standard JS library added to the projects

jQuery Examples (jquery Learning Center)

```
$("#myId");  
$(".myClass");  
$("input[name='first_name']");  
$("#contents ul.people li");  
$("div.myClass, ul.people");  
$("a.external:first");  
$("tr:odd");  
// Select all input-like elements in a form (more on this below).  
$("#myForm :input");  
$("div:visible");  
// All except the first three divs.  
$("div:gt(2)");  
// All currently animated divs.  
$("div:animated");
```

jQuery – Common use cases

DOM Manipulation

- Simplifies the process of selecting, modifying, and manipulating HTML elements in the Document Object Model (DOM), allowing developers to easily change content, attributes, and styles.

Event Handling

- Provides a straightforward way to attach event handlers to elements, enabling developers to respond to user interactions such as clicks, mouse movements, and keyboard inputs.

AJAX Requests

- Makes it easy to perform asynchronous HTTP requests (AJAX), allowing developers to load data from a server without refreshing the page, enhancing user experience.

Animation and Effects

- Includes built-in methods for creating animations and effects, such as fading, sliding, and showing/hiding elements, which can be used to enhance the visual appeal of web applications.

Cross-Browser Compatibility

- Abstracts away many of the inconsistencies between different web browsers, providing a consistent API that works across various platforms and versions.

Chaining Methods

- Allows method chaining, enabling developers to perform multiple operations on the same set of elements in a single line of code, improving code readability and efficiency.

Plugins

- Has a rich ecosystem of plugins that extend its functionality, allowing developers to easily add features like sliders, modals, and form validation to their applications.

Bootstrap

A front-end framework providing a collection of pre-designed HTML, CSS, and JavaScript components.

A flexible grid system allowing the creation of layouts that adapt seamlessly to different screen sizes and devices.

The framework includes a wide range of reusable & customisable components (navigation bars, buttons, modals, forms, and alerts).

Easy customization through its Sass variables and built-in theming options.

Bootstrap



Checkout form

Below is an example form built entirely with Bootstrap's form controls. Each required form group has a validation state that can be triggered by attempting to submit the form without completing it.

Billing address

First name Last name

Username

Email (Optional)

Address

Address 2 (Optional)

Country State Zip

☐ Shipping address is the same as my billing address
☐ Save this information for next time

Payment

☒ Credit card
☐ Debit card
☐ PayPal

Name on card Credit card number

Full name as displayed on card

Expiration CVV

[Continue to checkout](#)

Your cart

Product name \$12
Brief description

Second product \$8
Brief description

Third item \$5
Brief description

Promo code EXAMPLECODE -\$5

Total (USD) \$20

Promo code

```
<!DOCTYPE html>
<html lang="en">
  <head>...</head>
  <body class="bg-light">
    <div class="container">
      <div class="py-5 text-center">...</div>
      <div class="row">
        <div class="col-md-4 order-md-2 mb-4">
          <h4 class="d-flex justify-content-between align-items-center mb-3">...</h4>
          <ul class="list-group mb-3">
            <li class="list-group-item d-flex justify-content-between lh-condensed">...</li>
            <li class="list-group-item d-flex justify-content-between lh-condensed">...</li>
            <li class="list-group-item d-flex justify-content-between lh-condensed">...</li>
            <li class="list-group-item d-flex justify-content-between bg-light">...</li>
          </ul>
          <form class="card p-2">
            <div class="input-group">
              <input type="text" class="form-control" placeholder="Promo code">
              <div class="input-group-append">...</div>
            </div>
          </form>
        <div class="col-md-8 order-md-1">...</div>
      </div>
    </div>
  </body>
</html>
```

JS Data-Apps – D3D.js

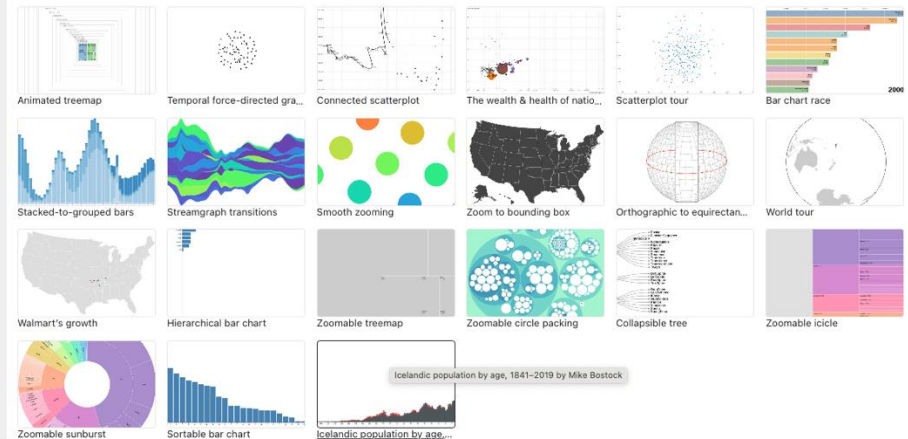
- JS library for interactive data visualisations
- Foundation of other libraries providing reusable charts:
 - C3
 - Plotly
- BSD licence

Gallery

Looking for a good D3 example? Here's a few (okay, 168...) to peruse.

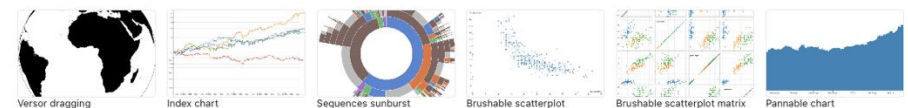
Animation

D3's [data join](#), [interpolators](#), and [easings](#) enable flexible [animated transitions](#) between views while preserving [object constancy](#).



Interaction

D3's low-level approach allows for performant incremental updates during interaction. And D3 supports popular interaction methods including [dragging](#), [brushing](#), and [zooming](#).



JS Charting – Plotly.js

- Charting library available for:
 - Python
 - JavaScript
 - R
 - Julia
- MIT license
- Build over D3

Plotly JavaScript Open Source Graphing Library

Built on top of [d3.js](#) and [stack.gl](#), Plotly.js is a high-level, declarative charting library. plotly.js ships with over 40 chart types, including 3D charts, statistical graphs, and SVG maps.

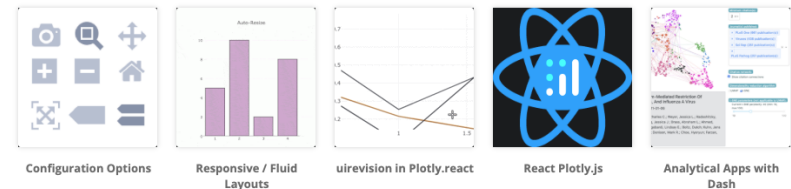
plotly.js is [free and open source](#) and you can [view the source](#), [report issues](#) or [contribute on GitHub](#).

Deploy Plotly.js AI Dash apps on private Kubernetes clusters: [Pricing](#) | [Demo](#) | [Overview](#) | [AI App Services](#)

[Read more about plotly.js features](#)

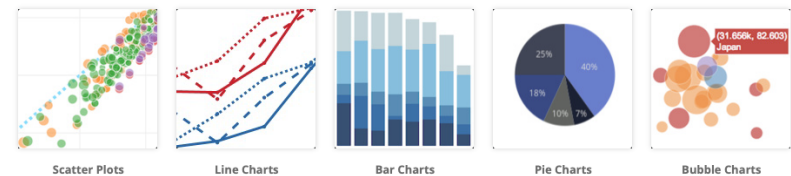
Fundamentals

[More Fundamentals »](#)



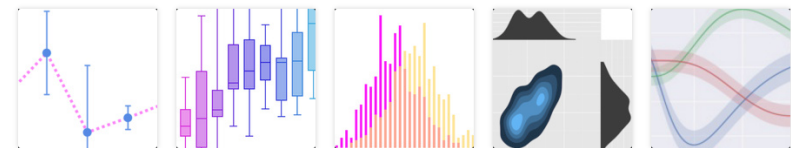
Basic Charts

[More Basic Charts »](#)



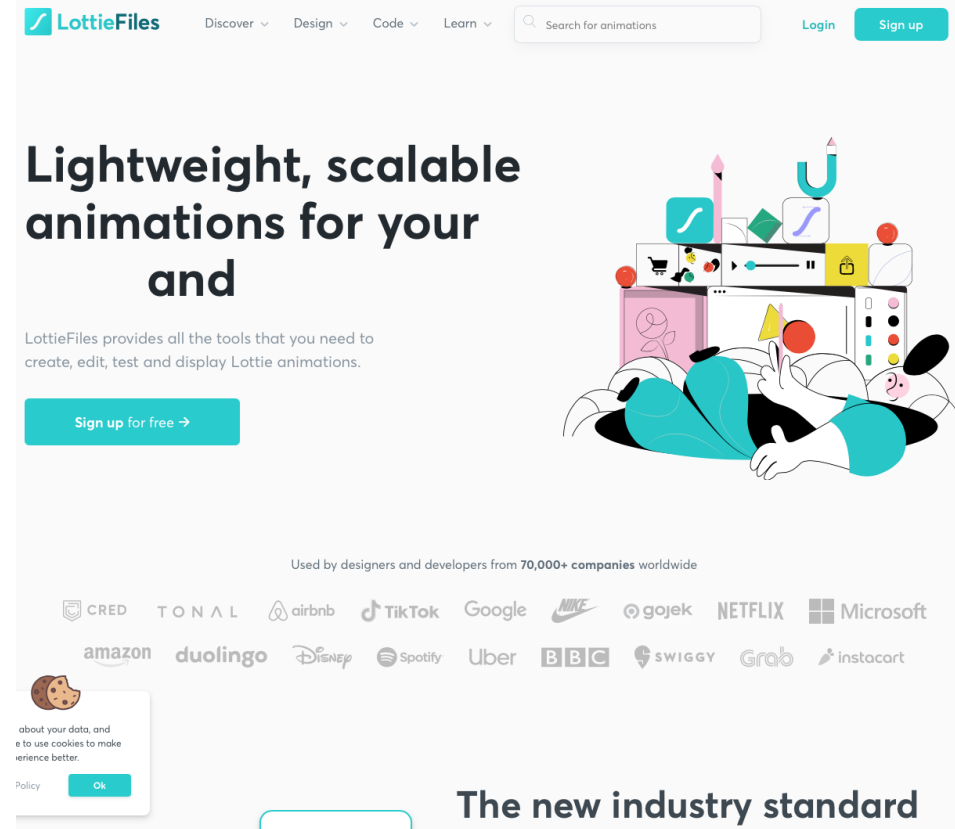
Statistical Charts

[More Statistical Charts »](#)



JS Animation – Lottie

- JS animation library developed by AirBnB
- Multiplatform solution – mobile, Web & native
- Renders Adobe AfterEffects animations
- Web – Lottie Editor
- Lottie Simple License (~copyleft)



JS Gaming – 2D Melon.js

- OS HTML-based game engine
- MIT license
- External libraries & tools (e.g. [Tiled map editor](#))
- 2D-based, with main focus on platform games – with other types possible

Player Killers' Exchange

Player Killers is an io game with elements of RPG. You'll be going through dangerous Dungeons killing monsters and opponents, turning a noble warrior from a skeleton. Kill other players and exchange their deaths for [nano](#), kill NPCs to collect weapons, equipment, and special items, collect blood samples to upgrade your character with transformation potions. Follow the directions at the top of your screen to find other players and objectives.

- For More information, and play the game click [here](#)



Roly-Polo

Roly-Polo is a fast and fun sport videogame for up to four players. In the game you control a PillBugBear (or a pair of them if you chose Twins-Mode), to win you have to score 15 points by throwing the purple ball through the ring goals.

It's available for Windows, Linux, Ouya and Android, it has support for gamepads, keyboard, and touchscreen, you can download the [FREE DEMO](#) and try it out.

- For More information, click [here](#)



The Covid-19 Game

Humanity is facing an unprecedented threat. The world is plagued by invisible enemies, namely, the unholy trinity of SARS, MERS & COVID-19. The people of the world are uniting like never before to battle the evil forces of the Viruses.

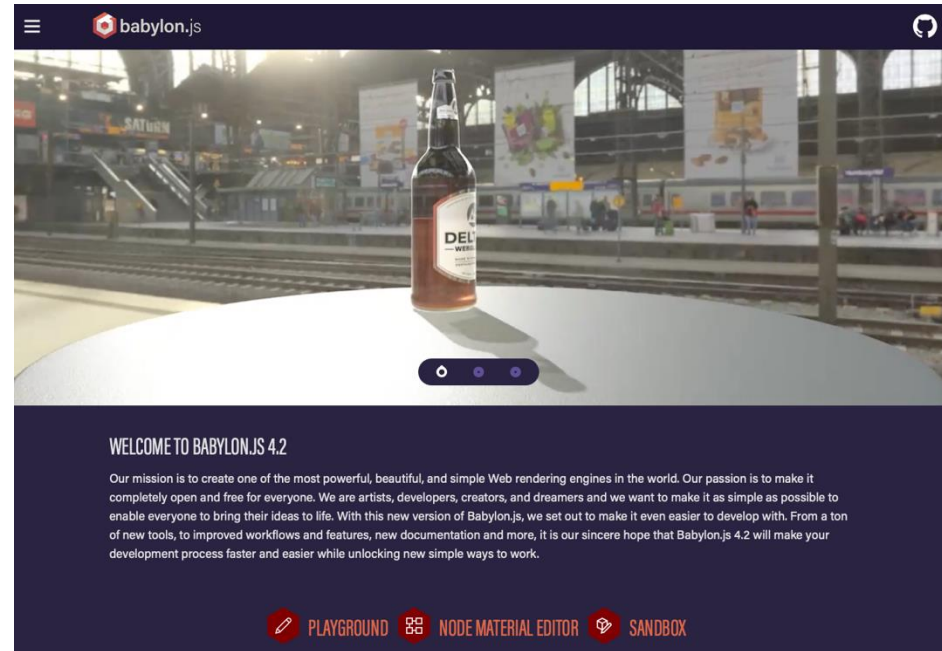
You are the leader of the red & white blood cells in the body. You must command the blood cells well to defeat the viruses of SARS, MERS & COVID-19. Your life depends on it! You will be given some time to prepare before the Viruses invade the body. Once they start invading the body, destroy all Viruses to win.

The COVID-19 Game is known as The Viral Game on Android due to



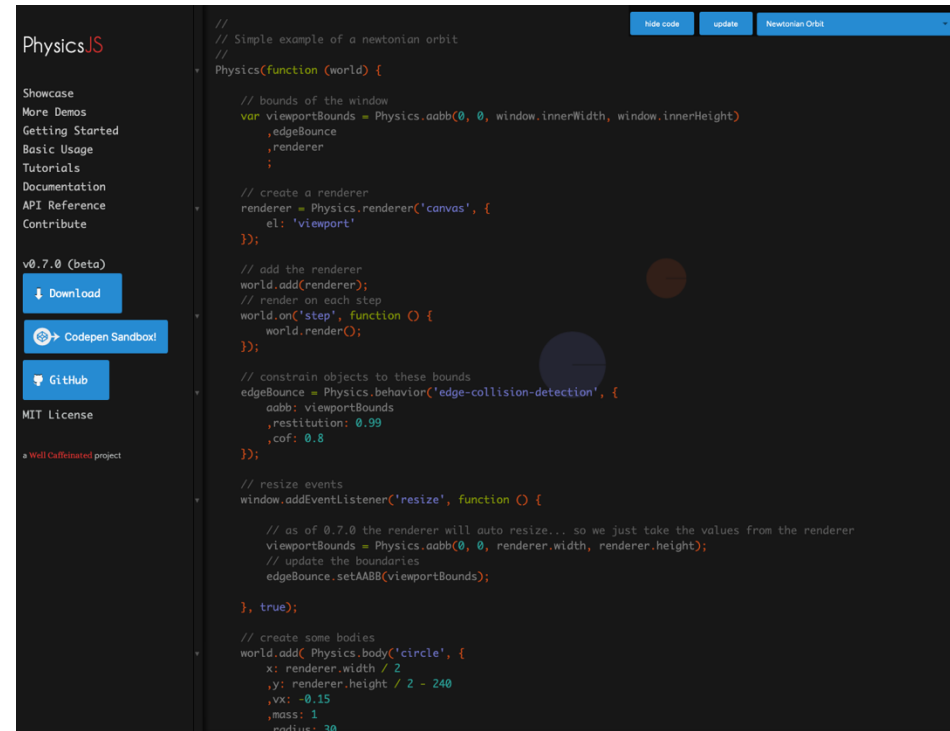
JS Gaming – 3D Babylon.js

- 3D rendering JS library
- WebGL-based API layer
- Data visualisation (no extra charting features)
- Web-oriented 3D presentations
- 3D games
- Apache License 2.0



JS Simulations – Physics.js

- Physics engine with basic visualisation
- Custom renderers possible (based on other libraries)
- Provides functionality to evaluate physical behaviour of defined objects
- MIT license



TensorFlow.js – AI&Web-app

A JS library allowing developers to define, train, and run machine learning models (browser or by Node.js).

A real-time inference on client-side applications for interactive experiences (e.g. image recognition, object detection or NLP).

A conversion of pre-trained TensorFlow models into a format that can be used in web applications.

WebGL for GPU acceleration, enhancing performance for computationally intensive tasks and enabling efficient execution of ML algorithms in the browser.

TF.js over TF

Browser-Based Execution

- TF.js enables machine learning models to run directly in the browser, providing faster response times and eliminating the need for server-side processing.

Accessibility

- Allows web developers to implement machine learning using JS, making it more accessible to those who may not be familiar with Python.

No Server Dependency

- TF.js enhances privacy by processing data locally in the browser, meaning sensitive information does not need to be sent to a server.

Interactivity

- The library facilitates the creation of interactive web applications that can respond to user inputs in real-time, enhancing user engagement.

Easy Model Deployment

- Developers can quickly deploy pre-trained TensorFlow models in web applications with minimal modifications, streamlining the integration process.

Cross-Platform Compatibility

- TF.js runs on various devices, including desktops, tablets, and mobile phones, ensuring a consistent experience across platforms.

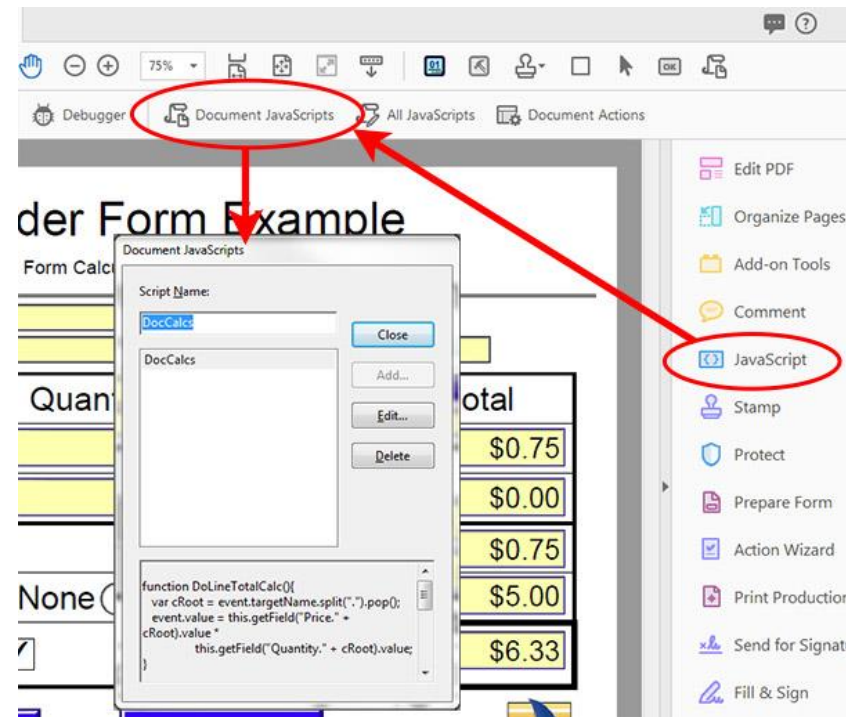
GPU Acceleration

- It utilizes WebGL for GPU acceleration, allowing for efficient execution of complex computations directly in the browser, improving performance for machine learning tasks.

JAVASCRIPT–BASED TECHNOLOGIES

JavaScript in PDFs

- JavaScript can be used in PDF forms
- Common usage of JS in PDFs is:
 - Population (calculation) of fields
 - Fields validation
 - Changing appearance
- Use tools provided by Adobe or use Apache PDActionJavaScript



Source: pdfscripting.com

TypeScript

- Microsoft extension of JavaScript
- Uses static typing
- Used for development
- Transcompiles (source-to-source) to JavaScript
- + class interfaces
- + generic programming
- + modules

```
interface User {  
  name: string;  
  id: number;  
}
```

```
class UserAccount {  
  name: string;  
  id: number;
```

```
  constructor(name: string, id: number) {  
    this.name = name;  
    this.id = id;  
  }  
}
```

```
const user: User = new  
  UserAccount("Murphy", 1);
```

CoffeeScript

- Another JS derivative
 - CS compiles to JS
 - Adds syntactic sugar inspired by
- Ruby, Python, and Haskell
 - Enhanced brevity and readability
 - Run on server-side

JavaScript

```
const mass = 72
const height = 1.78
const BMI = mass / height ** 2
if (18.5 <= BMI && BMI < 25) { alert('You are healthy!') }
```

CoffeeScript

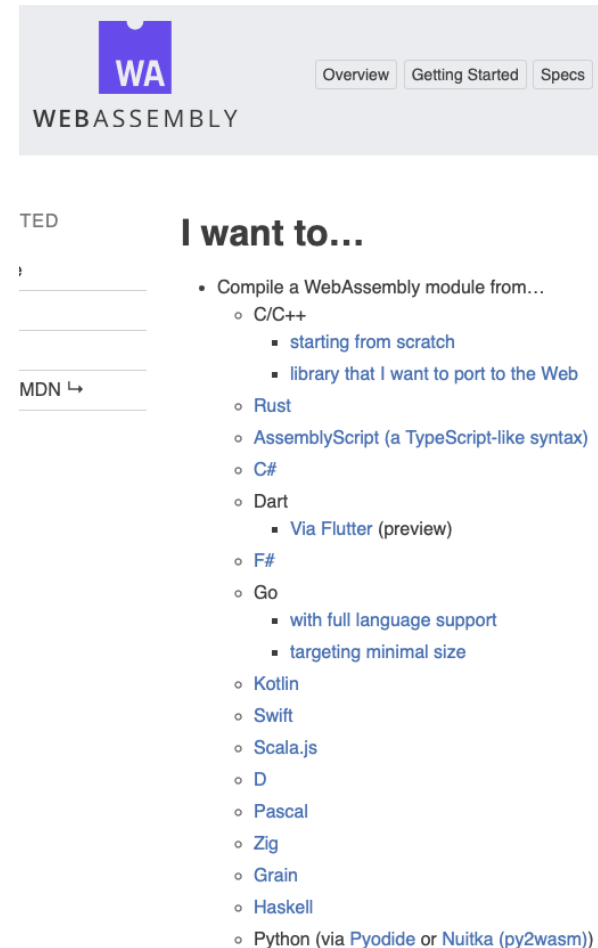
```
mass = 72
height = 1.78
BMI = mass / height**2
alert 'You are healthy!' if 18.5 <= BMI < 25
```

WebAssembly

- A portable binary-code and text format for executable programs enabling high-performance application to be run on web-pages (the core implementation)
- In theory could be run in other environments (Wasmtime or Wasmer standalones)
- Source code can be prepared in any language
- W3C recommendation in 2019
- Supported by all the popular browsers

WebAssembly development

- Think of Wasm as an equivalent of JVM's bytecode
- You can use many languages to write to source code (development phase)
- Wasm is a compiled form to be run (production phase)



WebAssembly – Stages

Source: Wikipedia

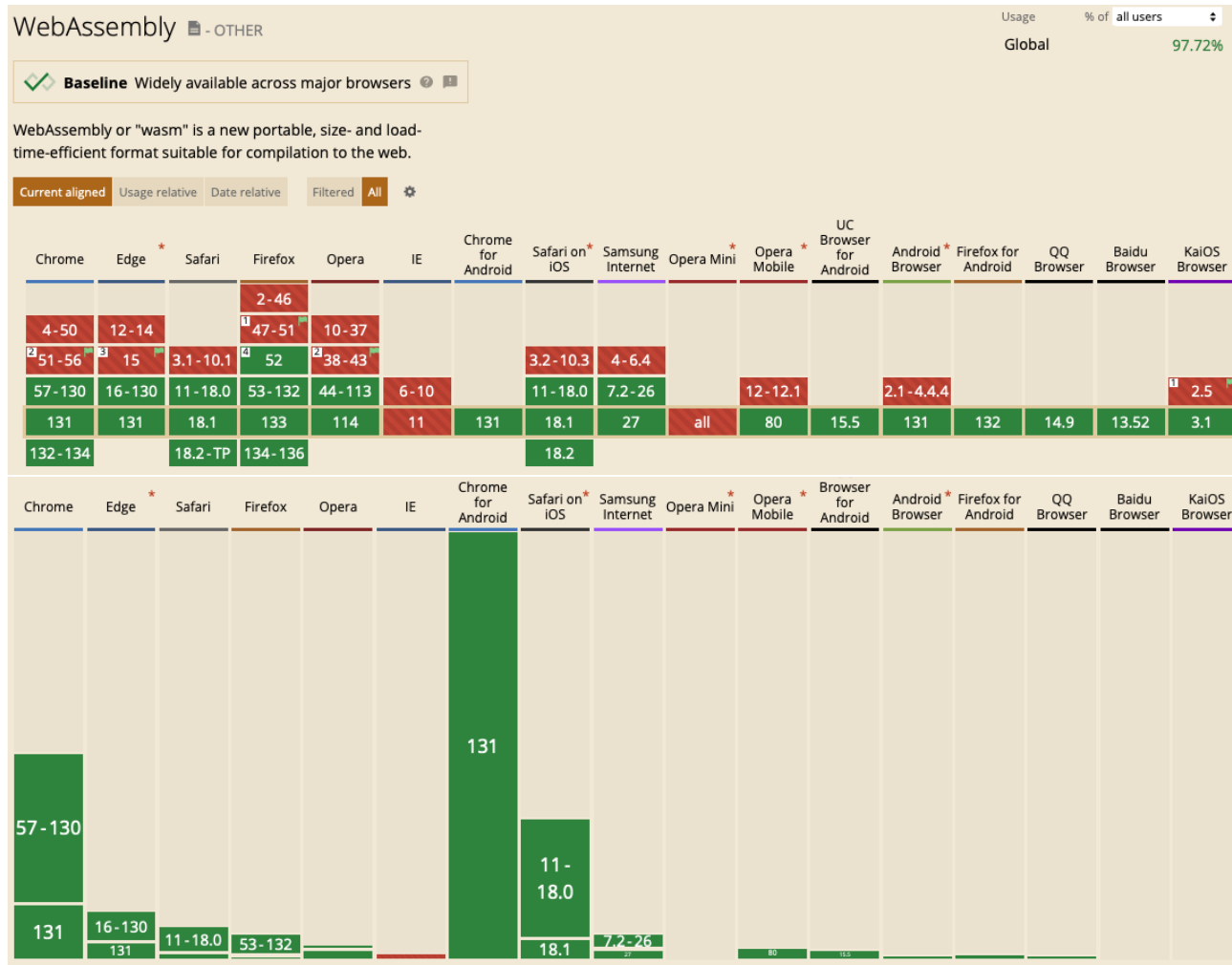
C

.wat

.wasm

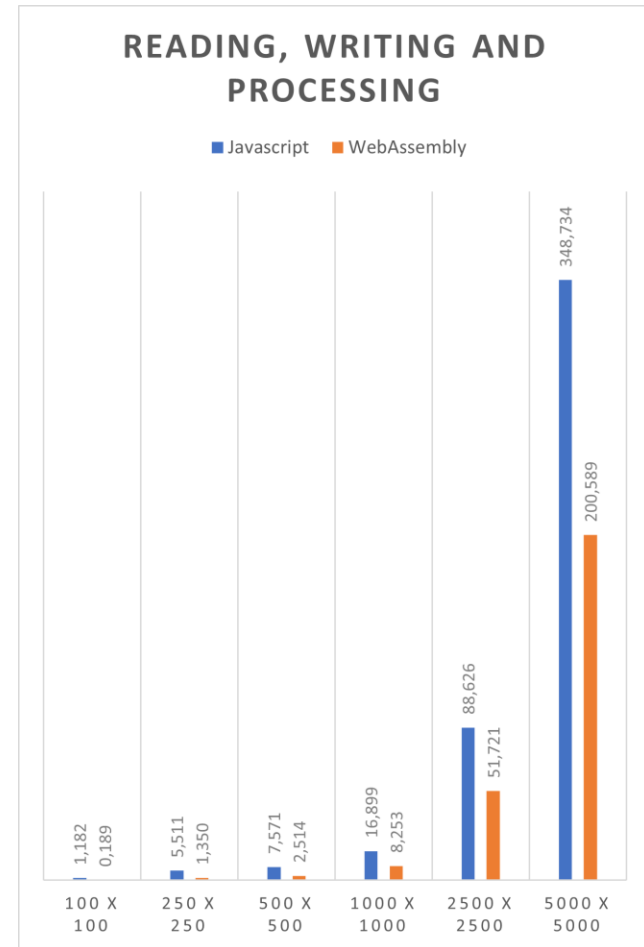
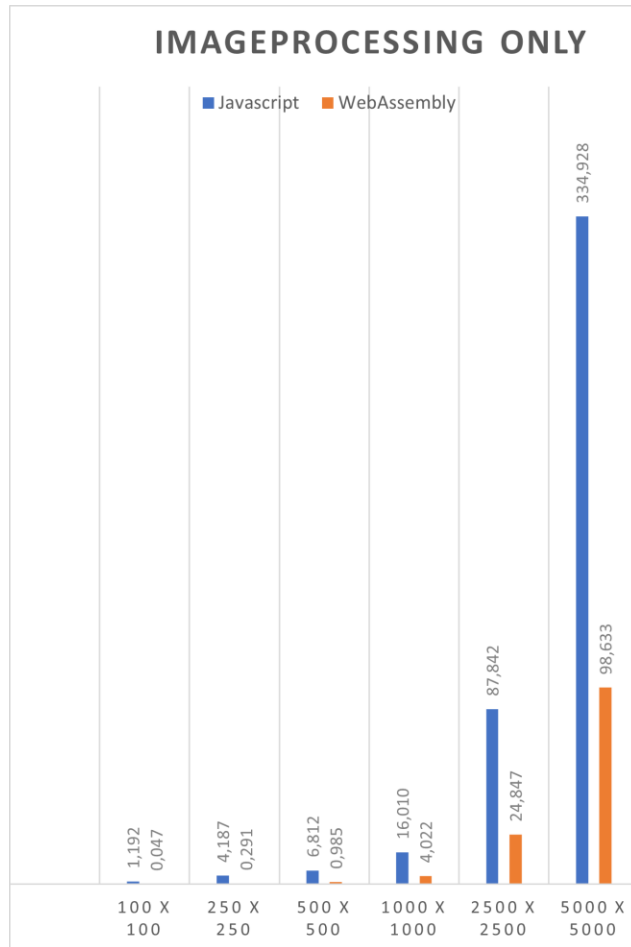
int factorial(int n) {	(func (param i64) (result	00 61 73 6D 01 00 00 00
if (n == 0)	i64)	01 06 01 60 01 7E 01 7E
return 1;	local.get 0	03 02 01 00
else	i64.eqz	0A 17 01
return n*factorial(n-1);	if (result i64)	15 00
}	i64.const 1	20 00
	else	50
	local.get 0	04 7E
	local.get 0	42 01
	i64.const 1	05
	i64.sub	20 00
	call 0	20 00
	i64.mul	42 01
	end)	7D
		10 00
		7E
		0B
		0B

Support for WebAssembly



WebAssembly performance

Sample test done by tmssoftware.com



AssemblyScript

- JS-derivative aimed at creation of WebAsm code
- Similar with TypeScript
- Gives low-level control of the code
- Gaining popularity (with WebAsm)

What are good use cases for AssemblyScript?

Computation-heavy logic like image manipulation, hot game logic, specialized algorithms, emulators, compilers and the likes are great use cases for WebAssembly, and as such for AssemblyScript as well. In some situations it may also be preferable to ship bytecode instead of minified JS, or just the ability to utilize a TypeScript-like language may open up new opportunities, for example for embedded scripting or plugins.

Dynamic Documents and Front-end Web Development

<https://takahirox.github.io/WebAssembly-benchmark/> - Google Chrome

Result (average [ms])

JavaScript: 542.7500

WebAssembly: 562.2100

JavaScript/WebAssembly: 0.9654

+ Test code

- JavaScript code

```
function jsCollisionDetection
```

Result (average [ms])

JavaScript: 305.4600

WebAssembly: 238.0200

JavaScript/WebAssembly: 1.2833

+ Test code

- JavaScript code

```
function jsQuicksortDouble(a:
```

Result (average [ms])

JavaScript: 1005.9200

WebAssembly: 761.8600

JavaScript/WebAssembly: 1.3203

+ Test code

- JavaScript code

```
function jsMultiplyDouble(a,
```

Result (average [ms])

JavaScript: 32.3400

WebAssembly: 22.7300

JavaScript/WebAssembly: 1.4228

+ Videos

+ Test code

- JavaScript code

```
function jsImageConvolute(data,
```

JavaScript and Other JS-oriented Technologies

Dynamic Documents and Front-end Web Development

<https://takahirox.github.io/WebAssembly-benchmark/> - Safari

Result (average [ms])

JavaScript: 1196.9000

WebAssembly: 621.5000

JavaScript/WebAssembly: 1.9258

+ Test code

- JavaScript code

```
function jsCollisionDetection(positions, radi
```

Result (average [ms])

JavaScript: 262.0000

WebAssembly: 229.2000

JavaScript/WebAssembly: 1.1431

+ Test code

- JavaScript code

```
function jsQuicksortDouble(array, start, enc
```

Result (average [ms])

JavaScript: 762.2000

WebAssembly: 757.4000

JavaScript/WebAssembly: 1.0063

+ Test code

- JavaScript code

```
function jsMultiplyDouble(a, b, n) {
```

Result (average [ms])

JavaScript: 55.1000

WebAssembly: 20.9000

JavaScript/WebAssembly: 2.6364

+ Videos

+ Test code

- JavaScript code

```
function jsImageConvolute(data, data2, wid
```

JavaScript and Other JS-oriented Technologies

WebAssembly – Usage cases

- Replace JS for computationally intensive tasks
- When no DOM manipulation is necessary
- Where there will be no constant communication with other JS pieces
- If memory intensive operations are to be done
- All above assume that the processing must be done on client side (due to information security, resource optimisation etc.)

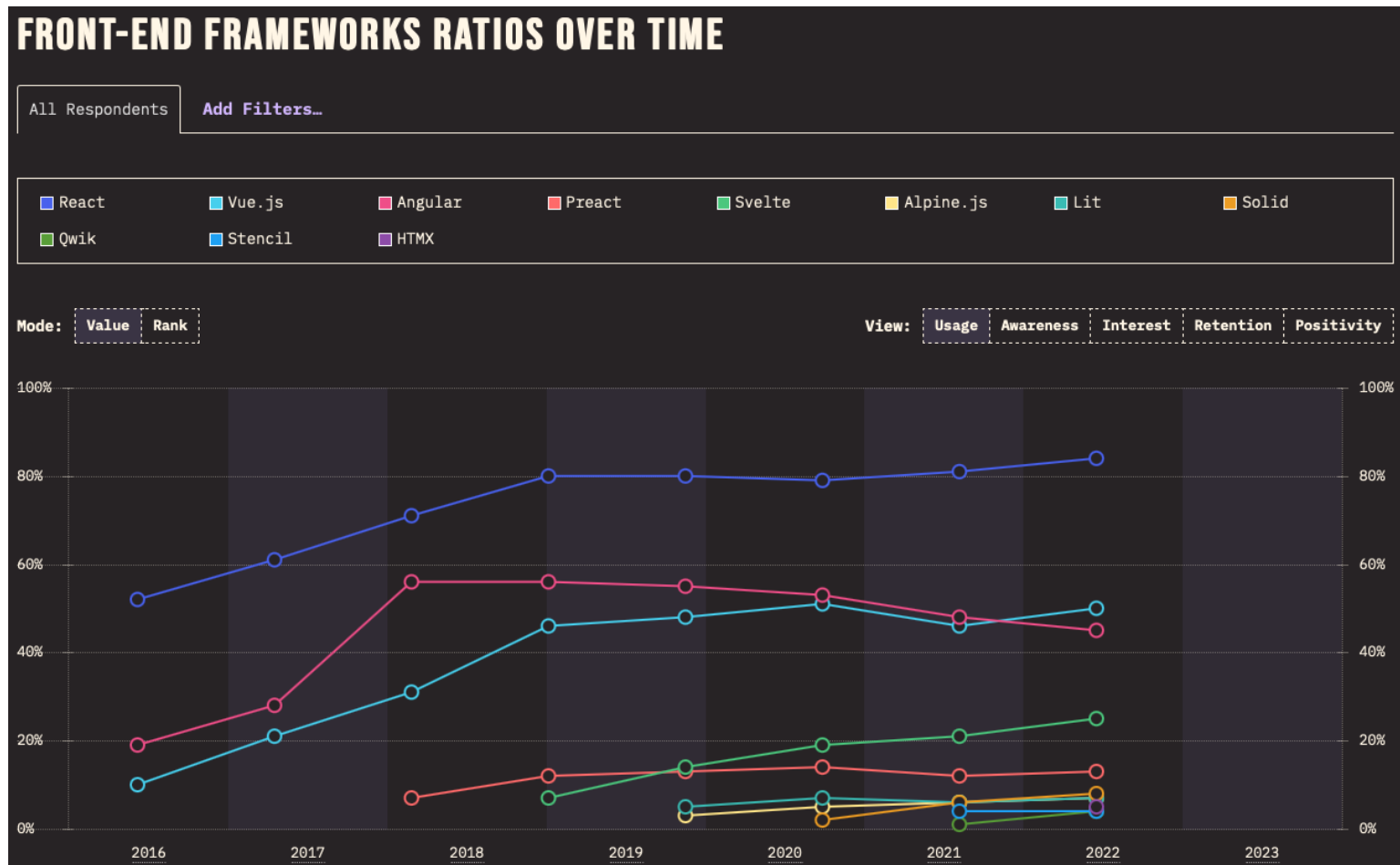
Dart (a part of Flutter)

- Google-lead project
- A part of Flutter UI SDK
- Delivers cross-platform applications
- Web, native or mobile native apps from a single code
- For Web code is transcompiled to JS

```
bool isEven(int x) {  
    // An if-else statement.  
    if (x % 2 == 0) {  
        return true;  
    } else {  
        return false;  
    }  
}  
  
List<int> getEvenNumbers(Iterable<int> numbers) {  
    var evenNumbers = <int>[];  
  
    // A for-in loop.  
    for (var i in numbers) {  
        // A single line if statement.  
        if (isEven(i)) {  
            evenNumbers.add(i);  
        }  
    }  
  
    return evenNumbers;  
}  
  
void main() {  
    var numbers = List.generate(10, (i) => i);  
    print(getEvenNumbers(numbers));  
}
```

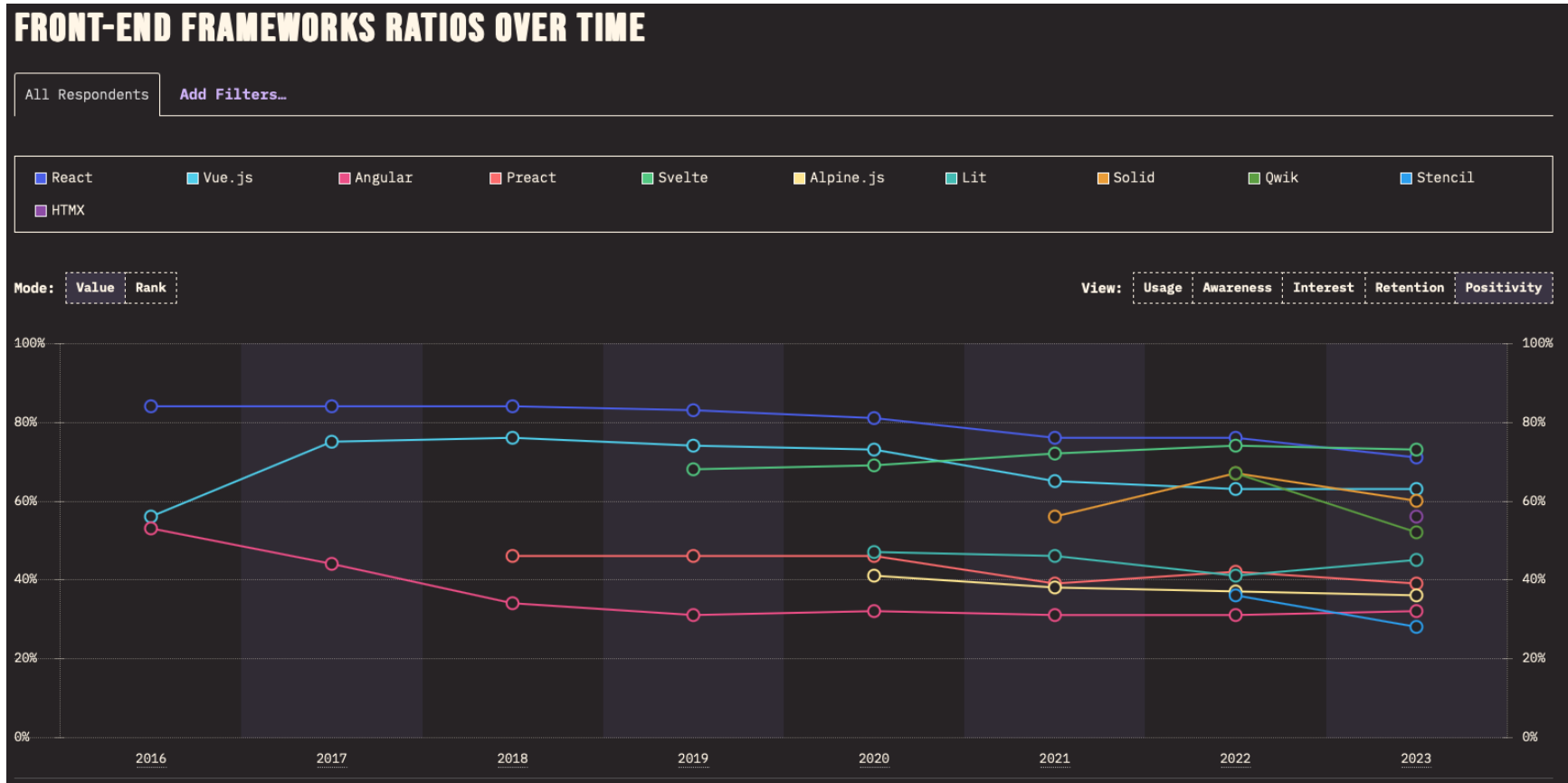
JavaScript Front-end Frameworks #1

<https://2023.stateofjs.com>



JavaScript Front-end Frameworks #2

<https://2023.stateofjs.com>



React #1

- Project started by Facebook
- Focus on single-page or mobile apps
- Framework limited to front-end (other aspects require additional libraries)
- Key element – JSX i.e. JavaScript XML – a mixture of JavaScript (action) and XML (content)

Dynamic Documents and Front-end Web Development

```
class Timer extends React.Component {
  constructor(props) {
    super(props);
    this.state = { seconds: 0 };
  }

  tick() {
    this.setState(state => ({
      seconds: state.seconds + 1
    }));
  }

  componentDidMount() {
    this.interval = setInterval(() => this.tick(), 1000);
  }

  componentWillUnmount() {
    clearInterval(this.interval);
  }

  render() {
    return (
      <div>
        Seconds: {this.state.seconds}
      </div>
    );
  }
}

ReactDOM.render(
  <Timer />,
  document.getElementById('timer-example')
);
```

RESULT

Seconds: 215

Angular #1

- Google-lead project of front-end web framework
- Single code generated to multiple platforms – web, mobile web & native mobile
- Based on TypeScript
- HTML syntax extended by templates and components

Dynamic Documents and Front-end Web Development

The screenshot shows a web development IDE with the following components:

- File Explorer (Left):** Displays the project structure, including files like `app.component.html`, `app.component.ts`, `app.module.ts`, and `hello-world-example.html`.
- Code Editor (Center):** Shows the content of `app.component.html`, which includes Angular directives like `<hello-world>`, `<hello-world-interpolation>`, `<hello-world-bindings>`, `<hello-world-template>`, `<hello-world-ngif>`, and `<hello-world-di>`.
- Terminal (Bottom):** Displays the output of the build process, including messages like "Compiled successfully" and "Browser application bundle generation complete."
- Browser Window (Right):** Shows the rendered application with the title "Understanding Angular". The page content includes:
 - "Hello World"
 - "This is my first component!"
 - "Hello, World!"
 - A button labeled "Trigger alert message"
 - "You can set my color in the component!"
 - "My color is blue"
 - "Hello World Template"
 - A button labeled "Make text editable!"
 - "I am read only!"
 - "Hello World: ngIf!"

JavaScript and Other JS-oriented Technologies

Node.js

- A JavaScript based runtime environment for back-end, running on V8 engine for event-driven web applications
- JavaScript app on server-side generates JavaScript to be send to a client
- Homogeneous technological stack
- Front-end developers may move to back-end and do not need server-specific language

Node.js – A basic web server

```
const http = require('http');

const hostname = '127.0.0.1';
const port = 3000;

const server = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');
  res.end('Hello World');
});

server.listen(port, hostname, () => {
  console.log(`Server running at http://${hostname}:${port}/`);
});
```

For a detailed comparison see: <https://www.netguru.com/glossary/node-js>

npm #1

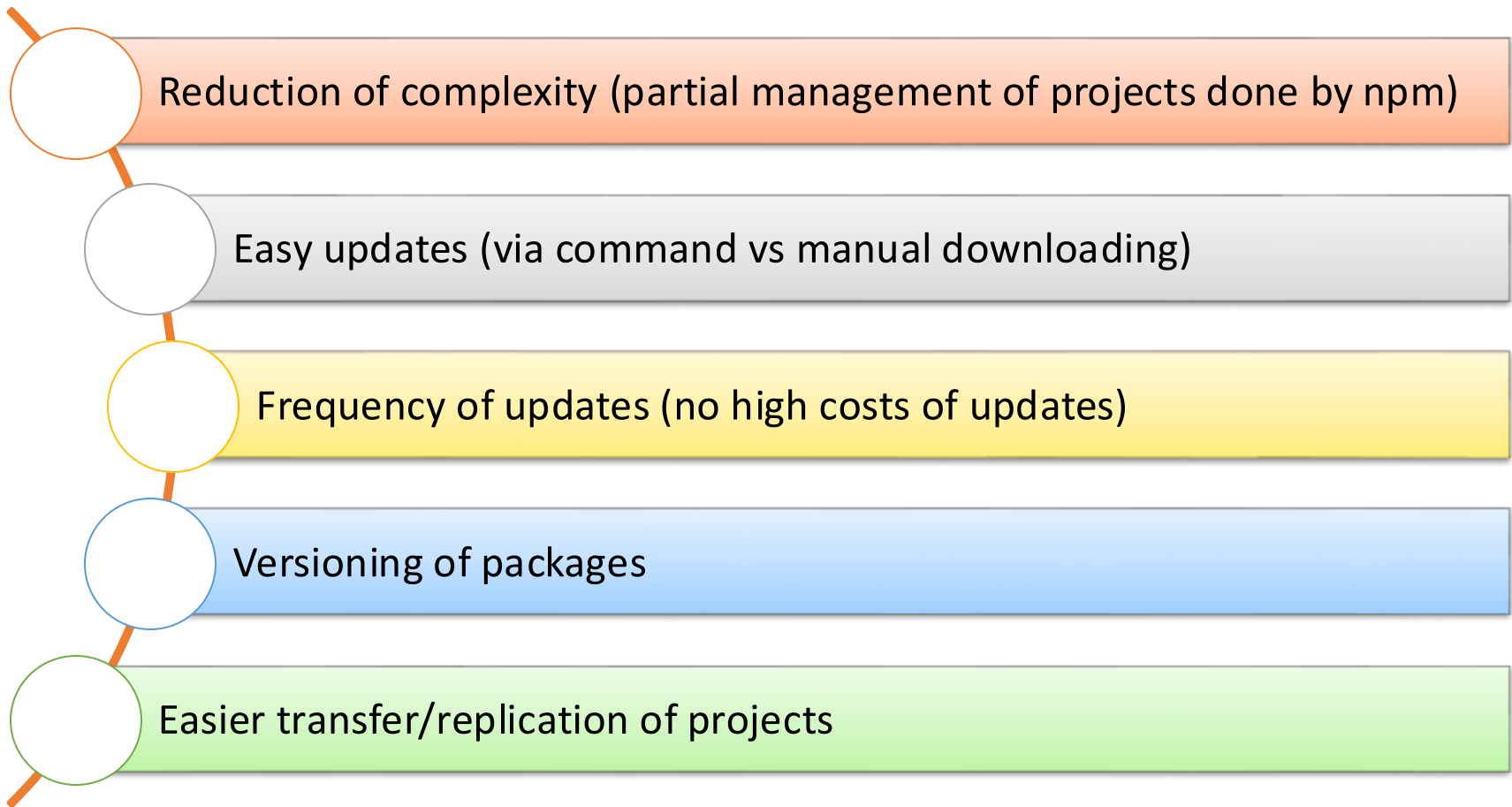
- npm is JavaScript package manager consisting three components:
 - Website (npmjs.com)
 - Command Line Interface (npm command)
 - Registry (a large public database of JS projects)
- Use npm to manage packages and ease the development of JS-based projects

npm #2

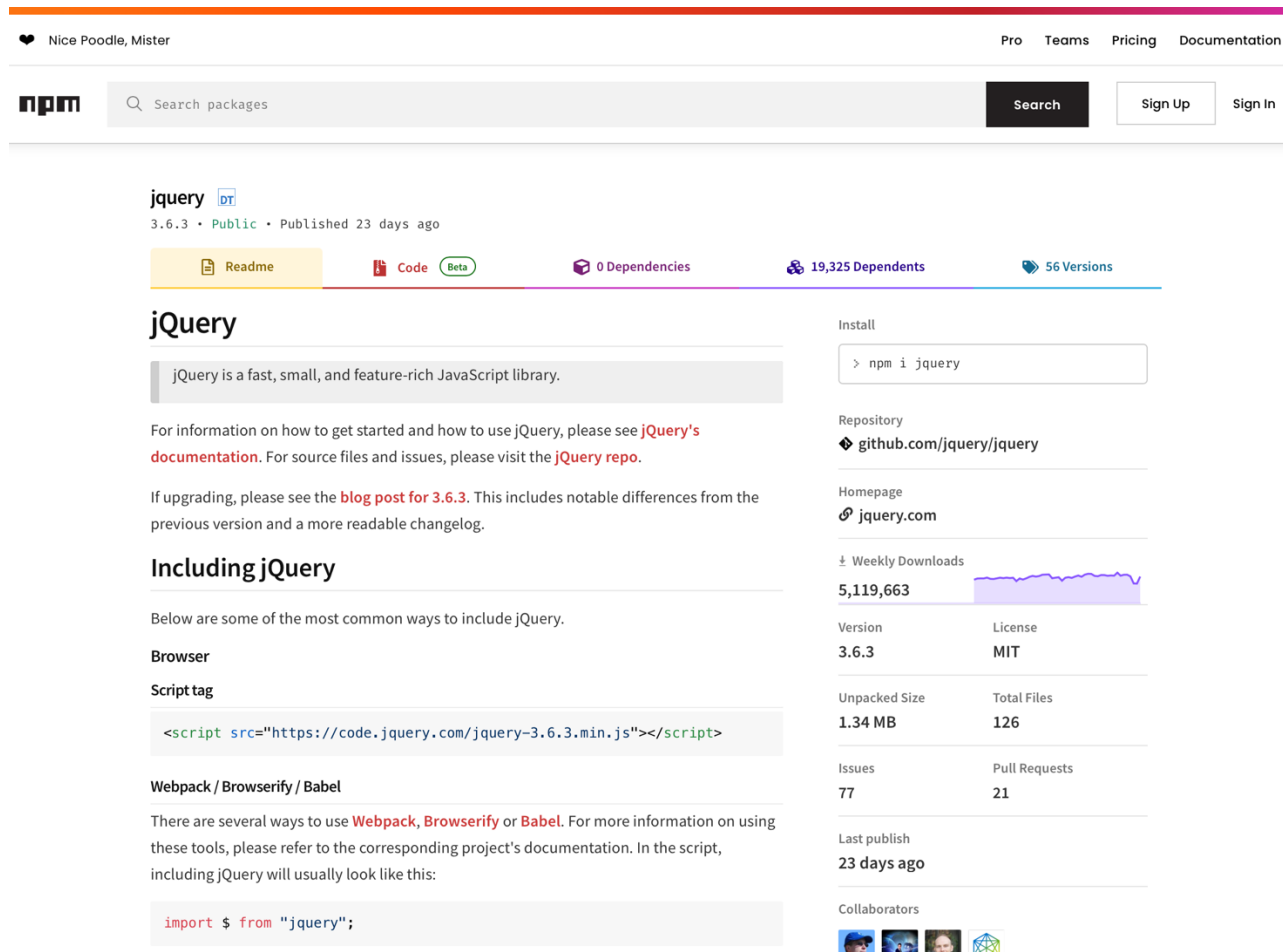
- Install jquery library
`npm install jquery`
- Search information about jquery
`npm search jquery`
- Explain chain of dependencies
`npm explain jquery`
- Open web documentation for a package
`npm docs jquery`

See <https://docs.npmjs.com/cli/v7>

npm #3



npm package example



The screenshot shows the npm package page for jQuery. At the top, there's a navigation bar with links for Pro, Teams, Pricing, and Documentation. Below this is a search bar with the text "Search packages" and buttons for "Search", "Sign Up", and "Sign In". The main content area features the jQuery package details. It includes the package name "jquery" with a "dt" icon, version "3.6.3", and status "Public". It also shows "Published 23 days ago". Below this are links for "Readme", "Code" (with a "Beta" badge), "0 Dependencies", "19,325 Dependents", and "56 Versions". The "jQuery" section describes it as a "fast, small, and feature-rich JavaScript library". It provides links to "jQuery's documentation" and the "jQuery repo". A "blog post for 3.6.3" is also mentioned. The "Including jQuery" section lists common ways to include jQuery: "Browser", "Script tag" (with a code snippet: `<script src="https://code.jquery.com/jquery-3.6.3.min.js"></script>`), and "Webpack / Browserify / Babel". It also mentions "There are several ways to use Webpack, Browserify or Babel. For more information on using these tools, please refer to the corresponding project's documentation. In the script, including jQuery will usually look like this:" followed by a code snippet: `import $ from "jquery";`. On the right side, there's an "Install" section with a code snippet: `> npm i jquery`. Below this is the "Repository" section with a link to "github.com/jquery/jquery". The "Homepage" section has a link to "jquery.com". The "Weekly Downloads" section shows a bar chart and the number "5,119,663". The "Version" section shows "3.6.3" and the "License" section shows "MIT". The "Unpacked Size" section shows "1.34 MB" and the "Total Files" section shows "126". The "Issues" section shows "77" and the "Pull Requests" section shows "21". The "Last publish" section shows "23 days ago". The "Collaborators" section shows four profile pictures.

Nice Poodle, Mister

Pro Teams Pricing Documentation

npm Search packages Search Sign Up Sign In

jquery 
3.6.3 • Public • Published 23 days ago

[Readme](#) [Code](#) [0 Dependencies](#) [19,325 Dependents](#) [56 Versions](#)

jQuery

jQuery is a fast, small, and feature-rich JavaScript library.

For information on how to get started and how to use jQuery, please see [jQuery's documentation](#). For source files and issues, please visit the [jQuery repo](#).

If upgrading, please see the [blog post for 3.6.3](#). This includes notable differences from the previous version and a more readable changelog.

Including jQuery

Below are some of the most common ways to include jQuery.

Browser

Script tag

```
<script src="https://code.jquery.com/jquery-3.6.3.min.js"></script>
```

Webpack / Browserify / Babel

There are several ways to use [Webpack](#), [Browserify](#) or [Babel](#). For more information on using these tools, please refer to the corresponding project's documentation. In the script, including jQuery will usually look like this:

```
import $ from "jquery";
```

Install

```
> npm i jquery
```

Repository

[github.com/jquery/jquery](#)

Homepage

[jquery.com](#)

Weekly Downloads

5,119,663

Version	License
3.6.3	MIT

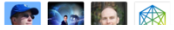
Unpacked Size	Total Files
1.34 MB	126

Issues	Pull Requests
77	21

Last publish

23 days ago

Collaborators



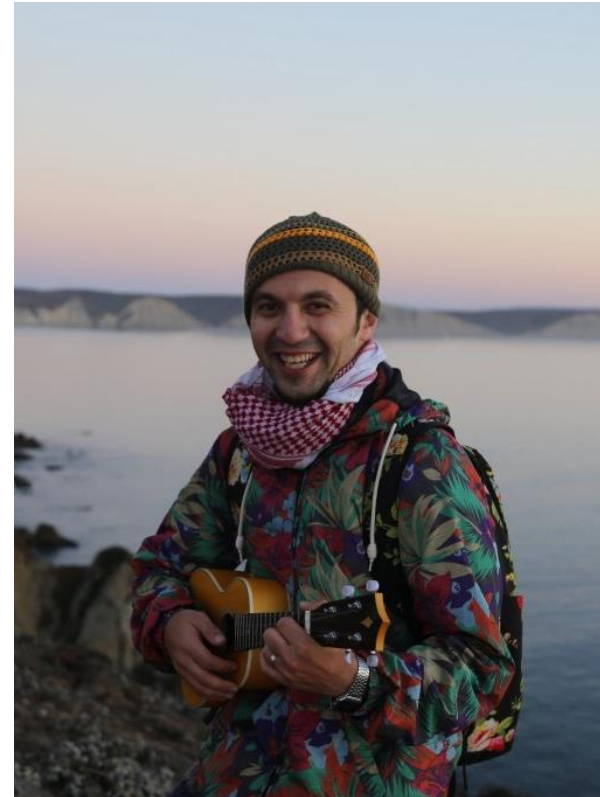
left-pad - Kik's victim

A general software development warning/lesson

```
use strict';
module.exports = leftPad;

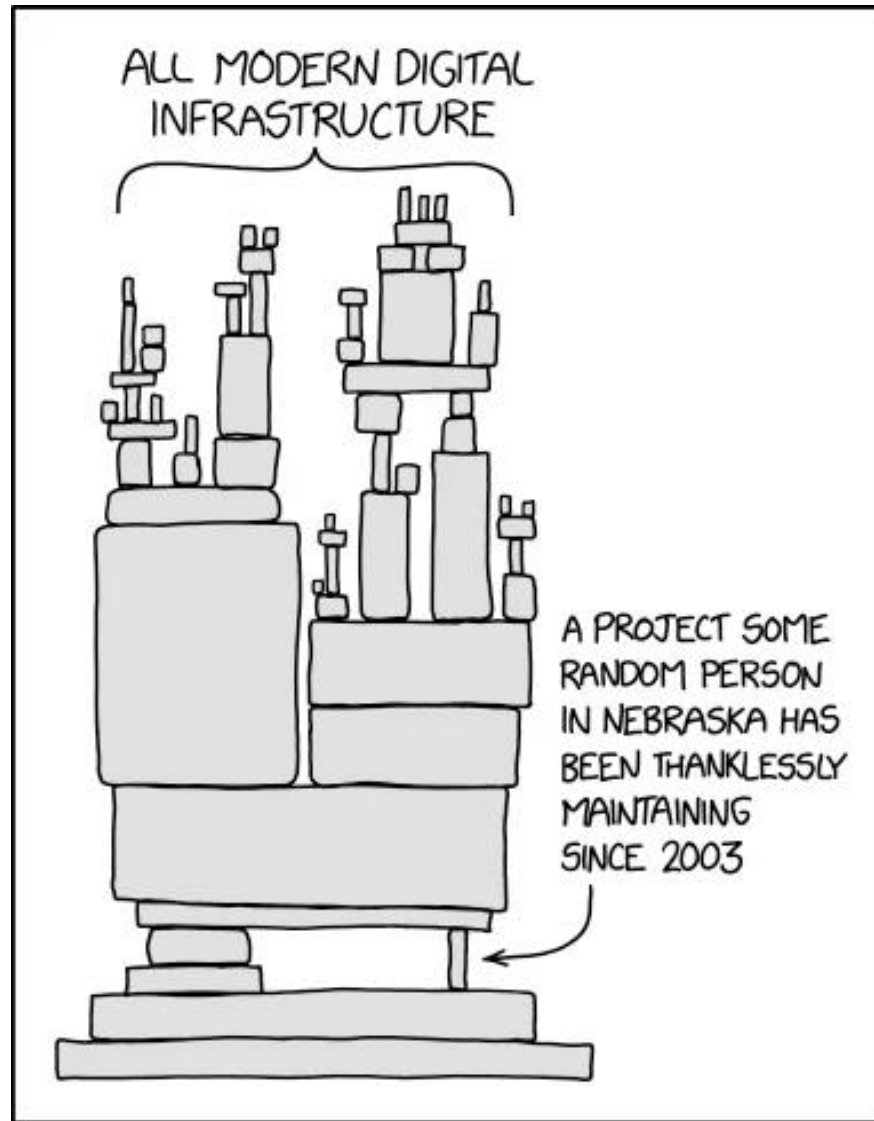
var cache = [
    '\n',
    '\n\n',
    '\n\n\n',
    '\n\n\n\n',
    '\n\n\n\n\n',
    '\n\n\n\n\n\n',
    '\n\n\n\n\n\n\n',
    '\n\n\n\n\n\n\n\n',
    '\n\n\n\n\n\n\n\n\n',
    '\n\n\n\n\n\n\n\n\n\n',
    '\n\n\n\n\n\n\n\n\n\n\n',
    '\n\n\n\n\n\n\n\n\n\n\n\n',
    '\n\n\n\n\n\n\n\n\n\n\n\n\n',
    '\n\n\n\n\n\n\n\n\n\n\n\n\n\n',
    '\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n',
    '\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n',
    '\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n',
    '\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n',
    '\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n',
    '\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n',
    '\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n'
];

function leftPad(str, len, ch) {
    // convert `str` to a `string`
    str = str + '';
    // `len` is the `pad`'s length now
    len = len - str.length;
    // doesn't need to pad
    if (len <= 0) return str;
    // `ch` defaults to ` `
    if (!ch || ch !== 0) ch = ' ';
    // convert `ch` to a `string` cuz it could be a number
    ch = ch + '';
    // cache common use cases
    if (ch === ' ' && len < 10) return cache[len] + str;
    // `pad` starts with an empty string
    var pad = '';
    // loop
    while (true) {
        // add `ch` to `pad` if `len` is odd
        if ((len & 1)) pad += ch;
        // divide `len` by 2, ditch the remainder
        len >>= 1;
        // "double" the `ch` so this operation count grows logarithmically on `len`
        // each time `ch` is "doubled", the `len` would need to be "doubled" too
        // similar to finding a value in binary search tree, hence O(log(n))
        if ((len) ch += ch;
        // `len` is 0, exit the loop
        else break;
    }
    // pad `str`!
    return pad + str;
}
```



Azer Koçulu

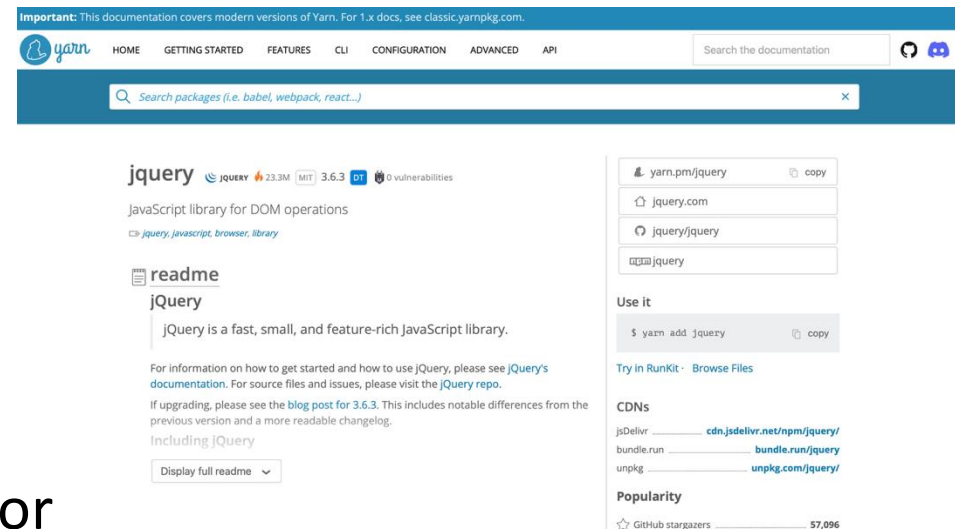
Dynamic Documents and Front-end Web Development



JavaScript and Other JS-oriented Technologies

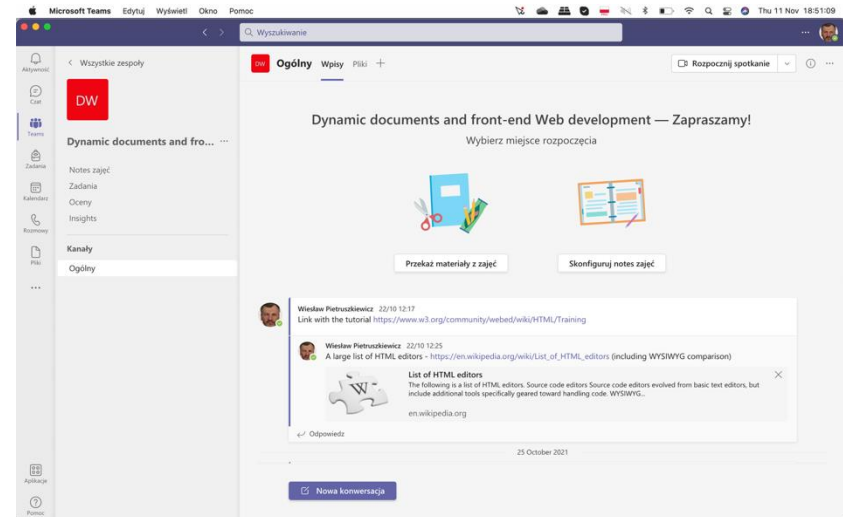
yarn – npm alternative

- yarn (<https://yarnpkg.com>) is alternative package manager for node.js
- yarn is concerned to be more secure (support of key IT companies)
- yarn installs dependencies in parallel (npm is sequential)
- npm is more popular (organic for node.js)
- See:
<https://www.geeksforgeeks.org/difference-between-npm-and-yarn/>



Electron.js – If server-side JS is not enough

- An OS framework to develop GUI applications with Web technologies
- Chromium V8 + Node.js
- Used by many apps:
 - Skype
 - MS Teams
 - Visual Studio Code



Summary

- JavaScript is a member of Web triad (HTML-CSS-JS)
- Its usage isn't limited to a client-side – Node.js creates a full JavaScript stack, keeping the same technology on both sides
- There are other languages derived from JS
- JavaScript – as a running platform – can be a result of compilation from other languages

TILL NEXT LECTURE